Team collaboration wall project (Mini Bulletin Board)

12562 – Aashik Niswin Robinson

The Team Collaboration Wall is a lightweight, web-based bulletin board application developed using ASP.NET Core, designed to enhance internal communication and collaboration within teams. It serves as a centralized space where team members can post updates, share announcements, and highlight achievements. This tool fosters transparency, encourages peer recognition, and keeps everyone aligned with ongoing activities and goals.

Team collaboration wall project (Mini Bulletin Board)	
WHAT?	**HOW?**
What are the modules required? 1. Admin Module 2. Manager Module 3. User Module (Employee Module) **Admin Module:** Admins have full control over the system and its configuration. **Key Features:** • Manage users and assign roles (Manager, Employee). • Create, Edit, or Delete Categories/Tags. • Approve or moderate employee posts.	**Admin Module:** **Login:** A. Admins can log in using their email and password. B. Admins can log in using their phone number. C. Admins can log in using username, password, email, and phone number. **Manage Users & Roles:** A. Use ASP.NET Identity (built-in user and role system) B. Create your own user table and role logic manually

• Moderate all posts and comments. • View system-wide analytics and engagement reports. **Manager Module** Managers oversee team-level activity and ensure communication is aligned. **Key Features:** • Pin important posts or announcements. • Create, edit and delete posts • Comment and react to posts. • Filter and view posts by department/project. • Generate weekly/monthly summaries for their team. **User Module (Employee)** Employees contribute to the wall and stay updated on team happenings. **Key Features:**	C. Use Google/Microsoft login and assign roles after login **Create, Edit, or Delete Categories/Tags:** A. Use a database table with a simple CRUD UI B. Hardcode categories/tags as enums in the backend C. Store categories/tags in a JSON config file **Content Moderation:** A. Allow All Users to Edit or Delete Any Post at Any Time B. Add a “Pending” status that requires Admin approval before publishing C. Send email to Admin every time a post is created for manual review **View system-wide analytics and engagement reports:** A. Build a dashboard with charts. B. Export raw data to Excel or CSV manually C. Send weekly email summaries with basic stats

• Create, edit and delete posts • Manage who can view the posts • Comment on and react to posts using emojis or tags • Filter posts by topic/project/department • Receive notifications for updates, replies, and mentions • Search and view archived posts	**Manager Module:** **Pin Important Posts or Announcements:** A. Add a IsPinned Boolean Field B. Create a Separate "PinnedPosts" Table C. Hardcode Pinned Posts in Frontend **Create, Edit, and Delete Posts:** A. Role-Based CRUD Operations B. Allow All Users Full CRUD Access C. Use Email-Based Post Submission **Comment and React to Posts:** A. Add Comment and Reaction Models B. Store Comments as JSON in Post Body C. Allow Anonymous Comments via External Form **Filter and View Posts by Department/Project:** A. Tag Posts with Metadata B. Create Separate Walls for Each Department C. Use Search Bar with Manual Keywords **Generate Weekly/Monthly Summaries for Their Team:** A. Scheduled Summary Reports via Backend Job

	B. Manual Export from Admin Dashboard C. Ask Users to Submit Their Own Weekly Reports **User Module:** **Create, Edit, and Delete Posts:** A. Role-Based CRUD with Ownership Check B. Allow All Users to Edit/Delete Any Post C. Submit Posts via Email to Admin **Manage Who Can View the Posts:** A. Visibility Settings per Post B. Use Static Groups with Hardcoded Access C. No Visibility Control—All Posts Are Public **Comment and React to Posts Using Emojis or Tags:** A. Separate Comment and Reaction Models B. Embed Comments as JSON in Post Body C. Allow Anonymous Comments via External Form

	Filter Posts by Topic/Project/Department: A. Tag Posts with Metadata B. Create Separate Walls for Each Category C. Use Search Bar with Manual Keywords **Receive Notifications for Updates, Replies, and Mentions:** A. Notification Table + SignalR Alerts B. Email-Only Notifications C. Manual Check—No Notification System **Search and View Archived Posts:** A. Status-Based Filtering with Search Indexing B. Move Archived Posts to Separate Table C. Delete Posts and Rely on Backups

WHY?	WHY NOT?
Admin Module: **Login:** A. Admins can log in using their email and password. This is the easiest and popular way to login because this avoids complexity	**Admin Module:** **Login:** B. Admins can log in using their phone number.

and provides a simple and secured way to login to the application. **Manage Users & Roles:** A. Use ASP.NET Identity (built-in user and role system) ASP.NET Identity is the best solution for managing users and roles because it provides a secure, built-in framework for authentication and authorization. It allows developers to easily assign roles like Admin or Employee and control access to features based on those roles. **Create, Edit, or Delete Categories/Tags:** A. Use a database table with a simple CRUD UI Using a database table with a simple CRUD UI is the best solution for managing categories and tags because it keeps the system structured, scalable, and user-friendly. Each category or tag can be stored as a record in a dedicated table, allowing easy creation, editing, and deletion through a clean interface.	While fast, this method lacks robust security and is vulnerable to SIM-based attacks. C. Admins can log in using username, password, email, and phone number. This adds unnecessary complexity and slows down the login process without improving security. **Manage Users & Roles:** B. Create your own user table and role logic manually This approach is time-consuming, error-prone, and lacks built-in security features like password hashing and token management. C. Use Google/Microsoft login and assign roles after login While convenient for authentication, it doesn't provide native role management or granular access control, requiring extra logic to securely assign and enforce roles post-login. **Create, Edit, or Delete Categories/Tags:** B. Hardcode categories/tags as enums in the backend Hardcoding categories/tags as enums limits flexibility, making it difficult to update or manage them without redeploying the application.

Content Moderation: B. Add a “Pending” status that requires Admin approval before publishing Adding a “Pending” status for content moderation is a smart and scalable solution that ensures quality control before posts go live. When an employee creates a post, it’s initially marked as Pending in the database. Admins can then review the post in a dedicated moderation queue and choose to either approve (changing status to Published) or reject it with feedback. **View system-wide analytics and engagement reports:** A. Build a dashboard with charts Building a dashboard with charts is the best solution for viewing system-wide analytics and engagement reports because it offers a clear, visual representation of key metrics. You can track post frequency, comment volume, reaction trends, and user activity across departments or projects. ## Manager Module: **Pin Important Posts or Announcements:** A. Add a IsPinned Boolean Field Adding an IsPinned boolean field to the Post model is a simple yet powerful	C. Store categories/tags in a JSON config file Storing them in a JSON config file lacks dynamic control and prevents real-time editing or tracking through a user interface. **Content Moderation:** B. Allow All Users to Edit or Delete Any Post at Any Time Allow All Users to Edit or Delete Any Post at Any Time undermines content integrity and accountability, leading to potential misuse and confusion. C. Send email to Admin every time a post is created for manual review Sending email to Admin every time a post is created for manual review is inefficient and unsustainable, especially in high-activity systems where automated moderation is more practical. **View system-wide analytics and engagement reports:** B. Export raw data to Excel or CSV manually Exporting raw data to Excel or CSV manually is tedious, error-prone, and lacks real-time insights or visual clarity for quick decision-making.

solution for enabling Managers to pin important posts or announcements. When IsPinned = true, the post is automatically prioritized in the UI—typically displayed at the top of the feed or in a highlighted section.This allows critical updates like deadlines, achievements, or alerts to remain visible regardless of post chronology. **Create, Edit, and Delete Posts:** A. Role-Based CRUD Operations Using Role-Based CRUD Operations is the best approach for managing post creation, editing, and deletion because it ensures that users can only perform actions appropriate to their role. For example, Employees can create and edit their own posts, Managers can moderate and edit posts within their team, and Admins have full control over all content. **Comment and React to Posts:** A. Add Comment and Reaction Models Adding Comment and Reaction models is the best solution for enabling users to interact with posts in a structured and scalable way. Each comment can be stored with metadata like author, timestamp, and post ID, allowing threaded discussions and moderation.	C. Send weekly email summaries with basic stats Sending weekly email summaries with basic stats offers limited interactivity and depth, making it hard to explore trends or drill into specific engagement metrics. **Manager Module:** **Pin Important Posts or Announcements:** B. Create a Separate “PinnedPosts” Table Creating a Separate “PinnedPosts” Table adds unnecessary complexity and duplication, making it harder to manage post relationships and updates. C. Hardcode Pinned Posts in Frontend Hardcoding Pinned Posts in Frontend is inflexible and requires code changes for every update, preventing dynamic control by managers. **Create, Edit, and Delete Posts:** B. Allow All Users Full CRUD Access Allowing all users full CRUD Access compromises content integrity and accountability, as it lets anyone modify or delete posts they didn’t create. C. Use Email-Based Post Submission Using Email-Based Post Submission lacks real-time interaction, making it hard to manage formatting, visibility,

Filter and View Posts by Department/Project:

A. Tag Posts with Metadata

Tagging posts with metadata is the best solution for filtering and viewing content by department or project because it keeps everything organized and scalable. Each post includes fields like DepartmentId and ProjectId, which link to reference tables.

Generate Weekly/Monthly Summaries for Their Team:

A. Scheduled Summary Reports via Backend Job

Using scheduled backend jobs to generate weekly or monthly team summaries is the most efficient and automated way to keep everyone informed.

User Module:

Create, Edit, and Delete Posts:

A. Role-Based CRUD with Ownership Check

Normal users (Employees) can create posts freely and are allowed to edit or delete only the ones they've authored. This ensures personal control over their own content while preventing changes to others' posts. The system checks both their role and ownership before allowing any action, keeping everything secure and fair.

Manage Who Can View the Posts:

A. Visibility Settings per Post

and immediate feedback within the platform.

Comment and React to Posts:

B. Store Comments as JSON in Post Body

Storing Comments as JSON in Post Body makes it difficult to query, moderate, or manage comments individually, limiting scalability and performance.

C. Allow Anonymous Comments via External Form

Allow Anonymous Comments via External Form risks spam, abuse, and lack of accountability, weakening trust and content quality within the platform.

Filter and View Posts by Department/Project:

B. Create Separate Walls for Each Department

Creating Separate Walls for Each Department fragments the user experience and makes cross-department collaboration harder, requiring users to switch views constantly.

C. Use Search Bar with Manual Keywords

Using Search Bar with Manual Keywords relies on inconsistent input and lacks structured filtering, making it difficult to surface relevant posts efficiently.

Generate Weekly/Monthly Summaries for Their Team:

Normal users (Employees) can control who sees their posts by selecting visibility settings like “Public,” or “Department Only” when creating or editing a post. These settings are stored as metadata and enforced by the system to ensure only authorized viewers can access the content. This gives employees flexibility while maintaining privacy and relevance across teams.

Comment and React to Posts Using Emojis or Tags:

A. Separate Comment and Reaction Models

To allow users to comment and react to posts using emojis or tags, we can implement separate Comment and Reaction models in the database. The Comment model stores text, author, timestamp, and post reference, enabling threaded discussions.

Filter Posts by Topic/Project/Department:

A. Tag Posts with Metadata

To let users filter posts by topic, project, or department, each post should be tagged with metadata fields like TopicId, ProjectId, and DepartmentId.

B. Manual Export from Admin Dashboard

Manual Export from Admin Dashboard is inefficient and prone to delays, as it relies on someone remembering to pull data and format it regularly.

C. Ask Users to Submit Their Own Weekly Reports

Asking Users to Submit Their Own Weekly Reports leads to inconsistent formats and missed submissions, making it hard to track team-wide performance reliably.

User Module:

Create, Edit, and Delete Posts:

B. Allow All Users to Edit/Delete Any Post

Allowing all Users to Edit/Delete Any Post breaks trust and accountability, as it lets users tamper with content they didn’t create, leading to confusion and misuse.

C. Submit Posts via Email to Admin

Submitting posts via Email to Admin is slow and disconnected from the platform, making post management inefficient and limiting user engagement.

Manage Who Can View the Posts:

B. Use Static Groups with Hardcoded Access

Using Static Groups with Hardcoded Access lacks flexibility and scalability,

Receive Notifications for Updates, Replies, and Mentions: A. Notification Table + SignalR Alerts To notify users about updates, replies, and mentions, use a Notification Table to store structured alerts and SignalR to deliver them in real time.	making it difficult to adapt permissions as teams evolve or new roles emerge. C. No Visibility Control—All Posts Are Public No Visibility Control—All Posts Are Public compromises privacy and relevance, exposing sensitive or department-specific content to unintended audiences.
Search and View Archived Posts: A. Status-Based Filtering with Search Indexing To enable users to search and view archived posts, implement status-based filtering where each post includes a Status field (e.g., Active, Archived). Archived posts are excluded from the main feed but remain searchable through a dedicated archive view.	**Comment and React to Posts Using Emojis or Tags:** B. Embed Comments as JSON in Post Body Embedding Comments as JSON in Post Body makes it hard to query, moderate, or update comments independently, reducing scalability and performance. C. Allow Anonymous Comments via External Form Allowing Anonymous Comments via External Form invites spam and abuse, undermining accountability and the quality of discussion within the platform. **Filter Posts by Topic/Project/Department:** B. Create Separate Walls for Each Category Creating Separate Walls for Each Category isolates content and reduces discoverability, making it harder for users to engage across topics or collaborate across departments.

	C.Use Search Bar with Manual Keywords Use Search Bar with Manual Keywords depends on inconsistent user input and lacks structured metadata, leading to poor filtering accuracy and missed relevant posts. **Receive Notifications for Updates, Replies, and Mentions:** B. Email-Only Notifications Email-Only Notifications are delayed and easy to miss, reducing real-time engagement and responsiveness within the platform. C. Manual Check—No Notification System Manual Check—No Notification System puts the burden on users to constantly monitor activity, leading to poor user experience and missed interactions. **Search and View Archived Posts:** B. Move Archived Posts to Separate Table Moving Archived Posts to Separate Table complicates data management and breaks continuity, making it harder to search or relate archived content to active posts. C.Delete Posts and Rely on Backups Delete Posts and Rely on Backups risk permanent data loss and delays in retrieval, undermining transparency and long-term content accessibility.

User Stories:

User Stories: Team collaboration wall project (Mini Bulletin Board)		
No.	**User Story**	**Acceptance Criteria**
US001	As a user, I want to log in with my credentials, so that I can access my role-specific dashboard.	• Form displays with username field, password field (masked), and prominent login button. • Input fields highlight when focused and display clear validation errors when empty. • Login activates via button click or Enter key with visible loading indicator during authentication • On successful login, user redirects to their role-appropriate dashboard. • Failed login displays "Invalid credentials" message without revealing which field is incorrect. • Form is fully keyboard navigable with proper focus management and screen reader compatibility. • "Forgot Password" link positioned below login form for easy access. • System enforces secure password handling throughout the authentication process. • Login page maintains the application's minimalist design aesthetic with appropriate spacing
US002	As a user, I want to create and share posts, so that I can communicate updates and ideas with my team.	• Post creation form includes title, body, optional tags, and visibility settings • Rich text editor supports basic formatting (bold, italic, lists, links) • “Post” button is disabled until required fields are filled • Posts are saved and displayed instantly on the dashboard • Visibility options include: public, department. Form displays validation errors for empty or invalid fields

		• Post creation is responsive across devices and accessible via keyboard • Confirmation message appears after successful submission • Posts are timestamped and show author name and role • Posts can be edited or deleted by the original author only • UI maintains consistent spacing and minimalist design
US003	As a manager, I want to pin important posts, so that my team sees critical updates first.	• “Pin” option available only to users with Manager role • Pinned posts appear at the top of the feed with a visual indicator • Only one post can be pinned per category at a time • Unpinning a post removes it from the top and restores chronological order • Confirmation prompt appears before pinning/unpinning • Pinned status persists across sessions and devices • Pinning action is logged for audit purposes • UI reflects pinned status with consistent styling
US004	As a user, I want to filter and search posts, so that I can find relevant content quickly.	• Filter options:department • Search bar supports keyword input with autocomplete • Combined filtering and search supported • Results ranked by relevance and timestamp • No results display fallback message • Fully responsive and accessible interface • Filters persist across sessions
		• Archived posts accessible via filter or tab

US005	As a user, I want to view archived posts, so that I can access older discussions and resources.	• Archived posts clearly labelled • Read-only view for archived content • Search includes archived posts • Responsive layout and accessibility support
US006	As a user, I want to tag department names with posts , so that I can share content with the right audience.	• Visibility options: public, department Selection available during post creation and editing • UI clearly displays visibility status
US007	As a user, I want to upload or change my profile picture, so that I can personalize my account and express my identity.	• Profile picture upload available in user profile • Drag-and-drop or file picker support • Real-time preview with cropping tool • Image format and size validation • Instant update across comments and posts • Responsive and accessible UI
US008	As a manager, I want to log in securely, so that I can access my dashboard and manage team content	• Same login flow as user with role-based redirection • Responsive and accessible interface
US009	As a manager, I want to pin important posts, so that my team sees critical updates first.	• “Pin” option available only to managers • Pinned posts appear at top with visual indicator • Unpin restores chronological order • Action logged for audit • Responsive and accessible UI

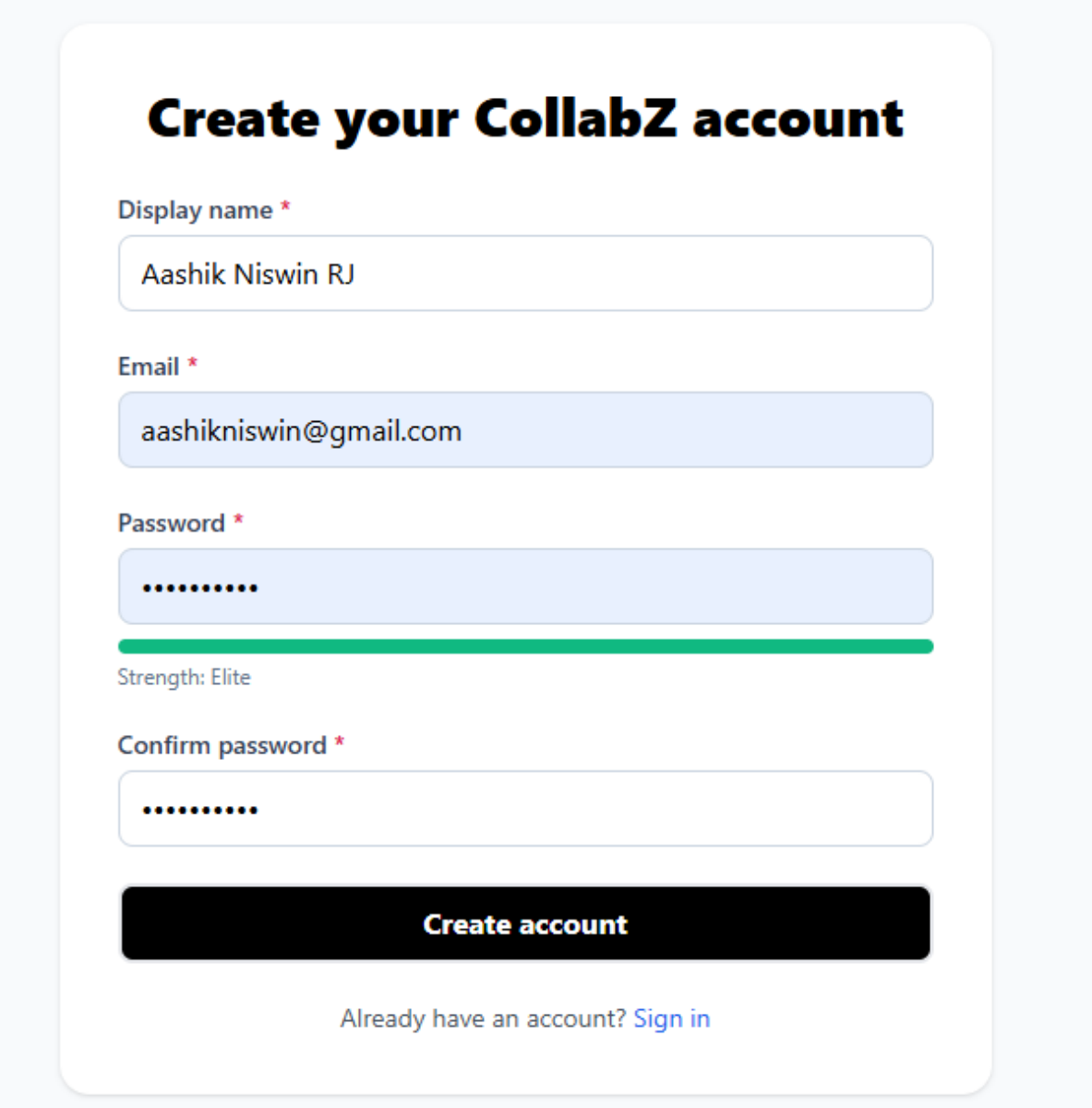
US010	As a manager, I want to view engagement metrics, so that I can assess participation and impact.	• Metrics: post views, comments, reactions per user • Filter by date, department, project • Graphs and tables with export option • Responsive and accessible dashboard
US011	As a manager, I want to generate weekly/monthly summaries, so that I can track progress and share updates.	• Summary includes top posts, engagement stats, key mentions • Exportable as PDF or CSV • Auto-generation with manual override • Responsive layout and accessibility support
US012	As a manager, I want to change my profile picture, so that my team can easily recognize me in communications and reports.	• Profile picture update option in account settings • Image cropping and preview functionality • Validates image format and dimensions • Updates synced across team views and reports • Confirmation message on successful update • Responsive and accessible UI
US013	As an admin, I want to log in securely, so that I can access moderation and analytics tools.	• Same login flow with admin role redirection • Admin dashboard shows pending approvals, analytics, user roles • Responsive and accessible interface

US014	As an admin, I want to approve posts before publishing, so that I can ensure content quality.	• Posts enter “Pending Approval” queue • Preview with metadata and comment field • Approve/Reject buttons with confirmation • Rejected posts notify author with feedback • Action logged with timestamp and admin ID • Bulk approval supported • Responsive and accessible UI
US015	As an admin, I want to manage user roles, so that I can control access across the platform	• Role assignment interface with search and filters • Roles: User, Manager, Admin • Changes logged with timestamp • Confirmation prompts for changes • Responsive and accessible design
US016	As an admin, I want to view system-wide analytics, so that I can monitor platform usage.	• Metrics: active users, post volume, comment count, reaction trends • Filters by date, department, role • Exportable graphs and tables • Real-time or scheduled refresh • Responsive and accessible dashboard
US018	As an admin, I want to archive or restore posts, so that I can manage long-term content visibility.	• Archive toggle available in post management view • Archived posts hidden from main feed • Restore returns post to original position • Action logged with timestamp • Responsive and accessible UI

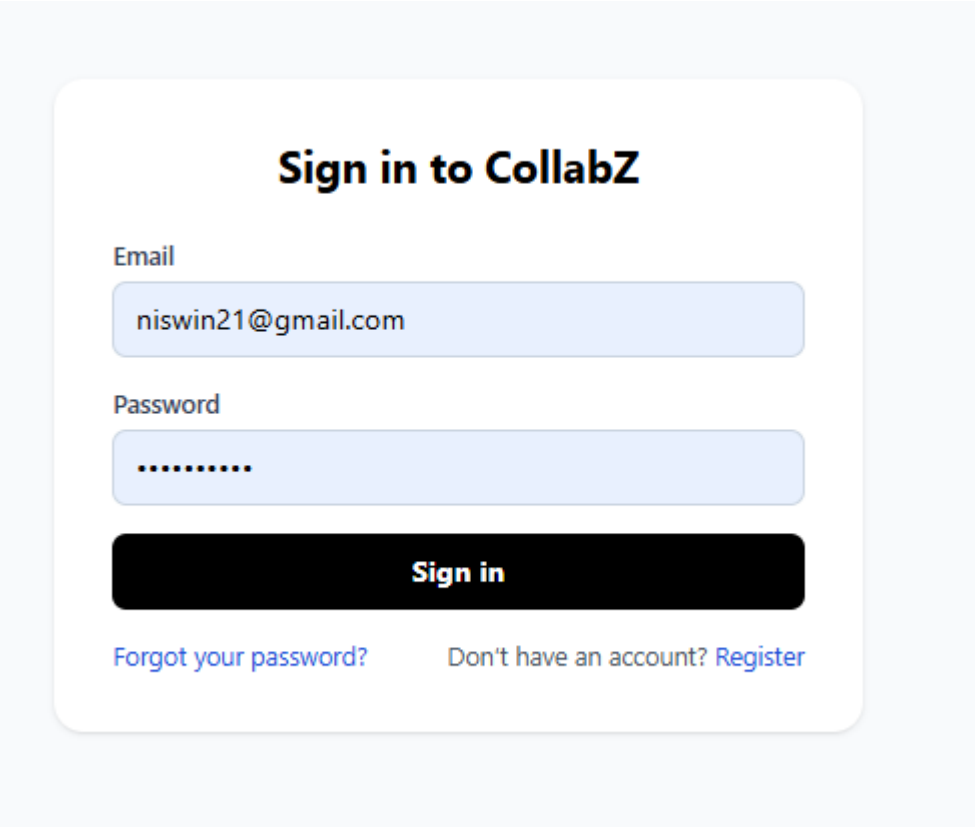
US019 US020	As an admin, I want to export platform data, so that I can generate reports or perform audits.	• Export options: CSV, PDF • Selectable data types: posts, comments, users, analytics • Confirmation prompt before export • File download initiated securely • Responsive and accessible interface
US021	As an admin, I want to update or change my profile picture, so that my identity is clearly represented across the platform.	• Profile picture upload available in admin settings • Supports image preview before saving • Accepts JPEG, PNG formats with size validation • Changes reflected instantly across admin dashboard • Action logged with timestamp for audit trail • Responsive and accessible UI

Project Implementation:

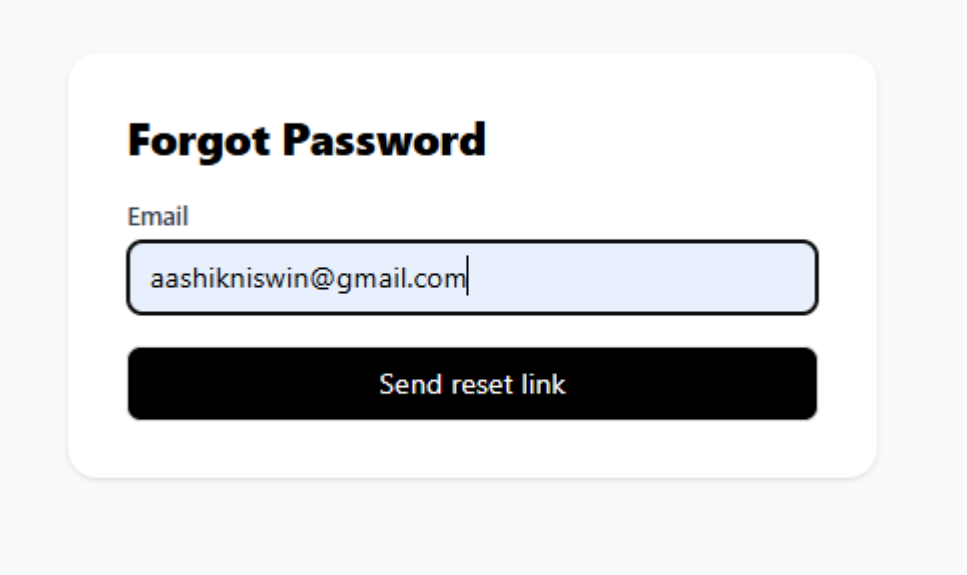
Register Page:

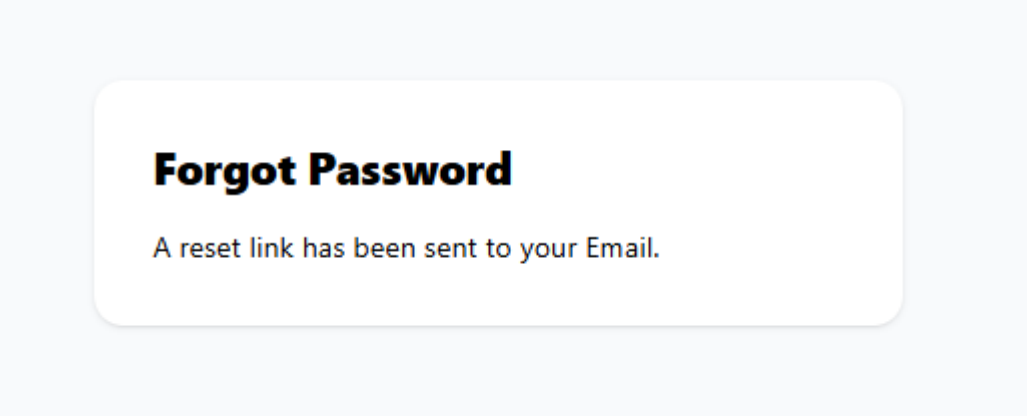


Login Page:

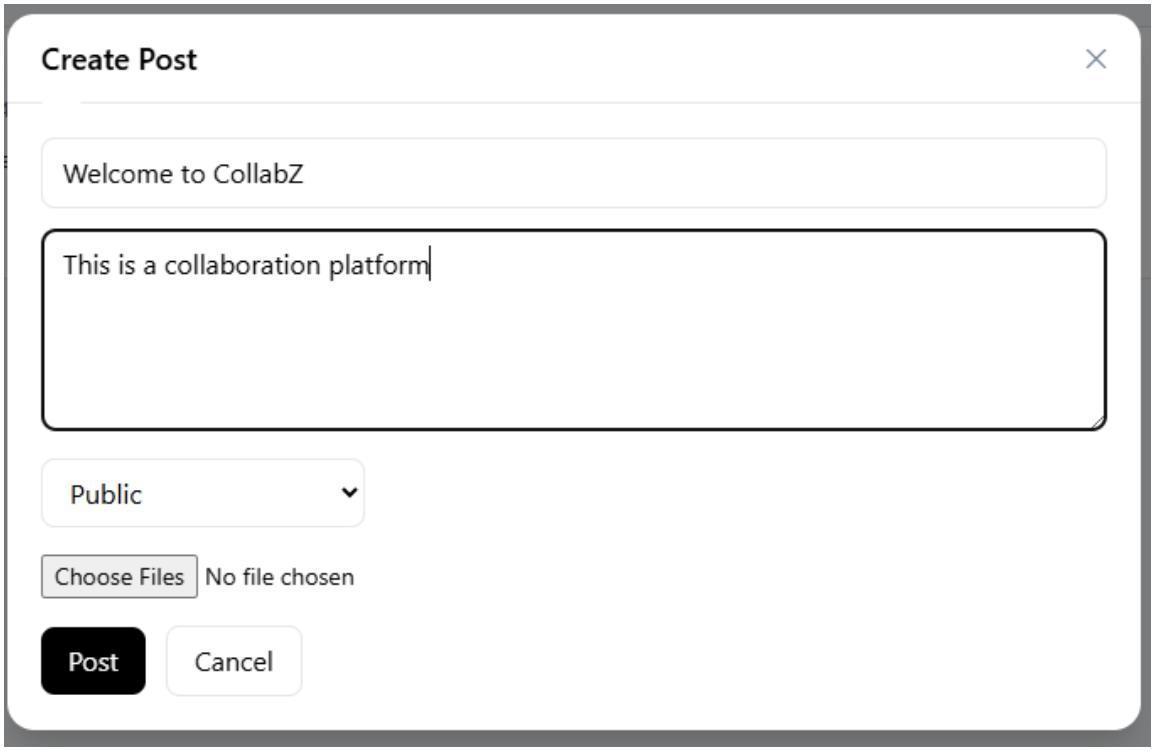


Forgot Password Page:

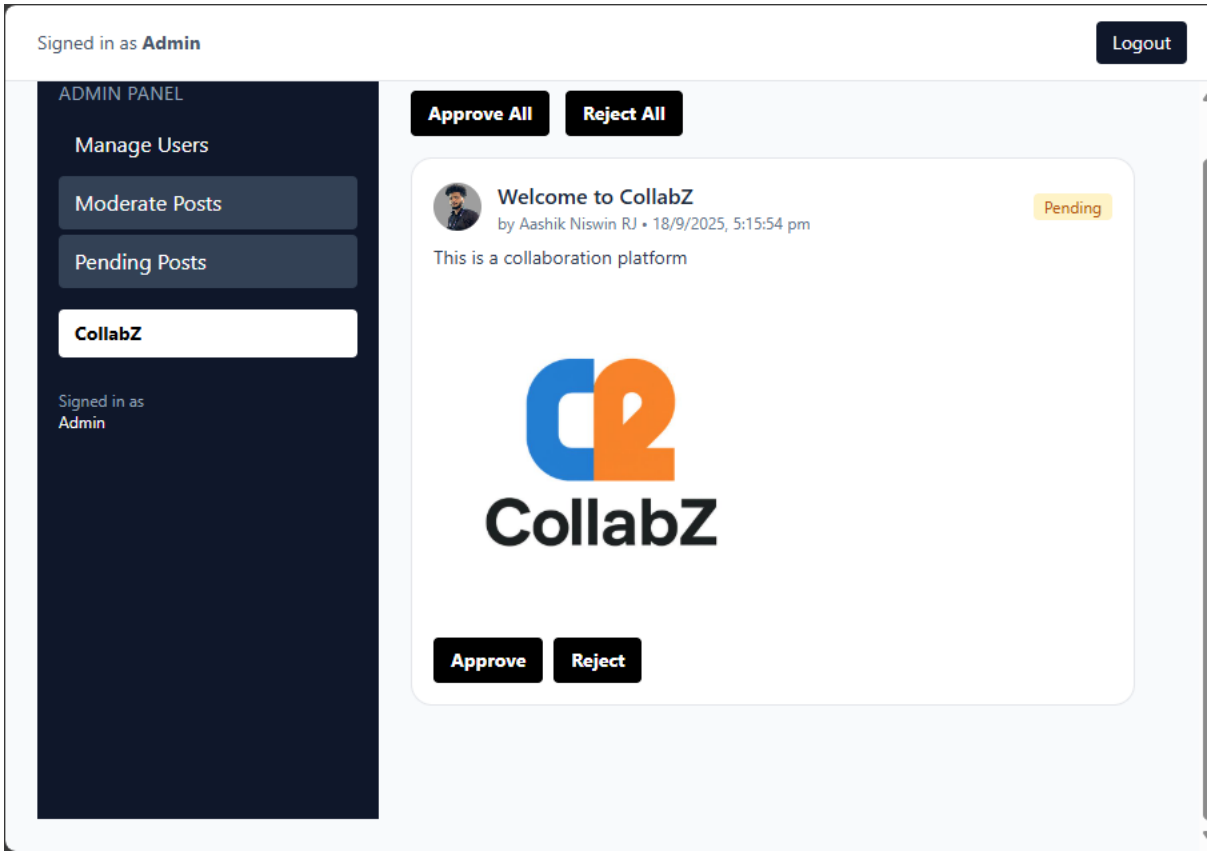

Forgot Password
Email
aashikniswin@gmail.com
Send reset link


Forgot Password
A reset link has been sent to your Email.

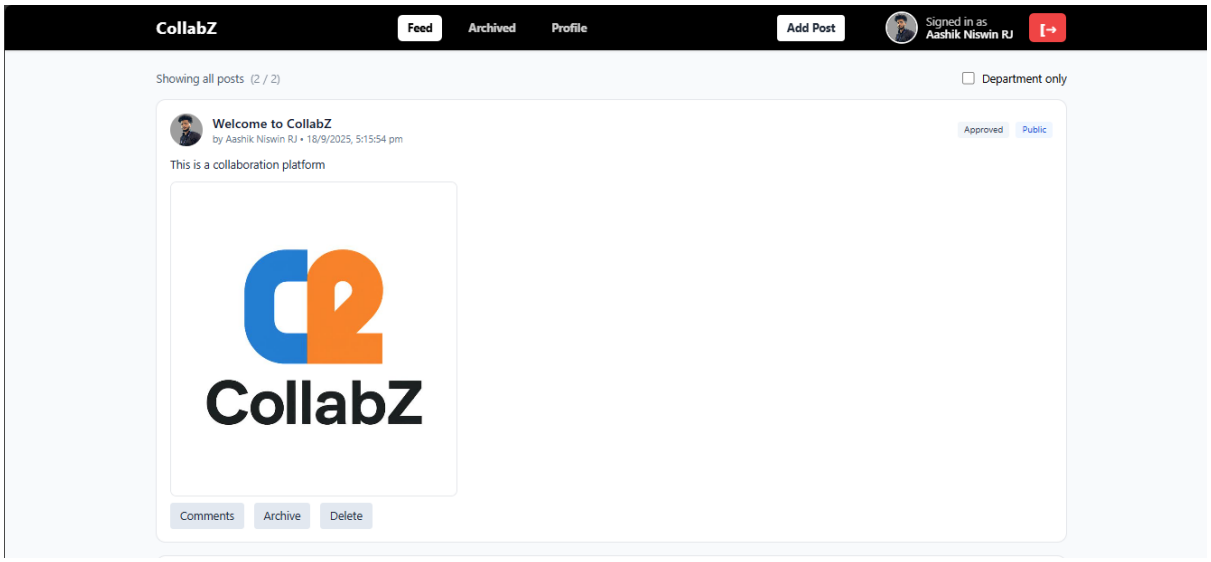
Add Post:



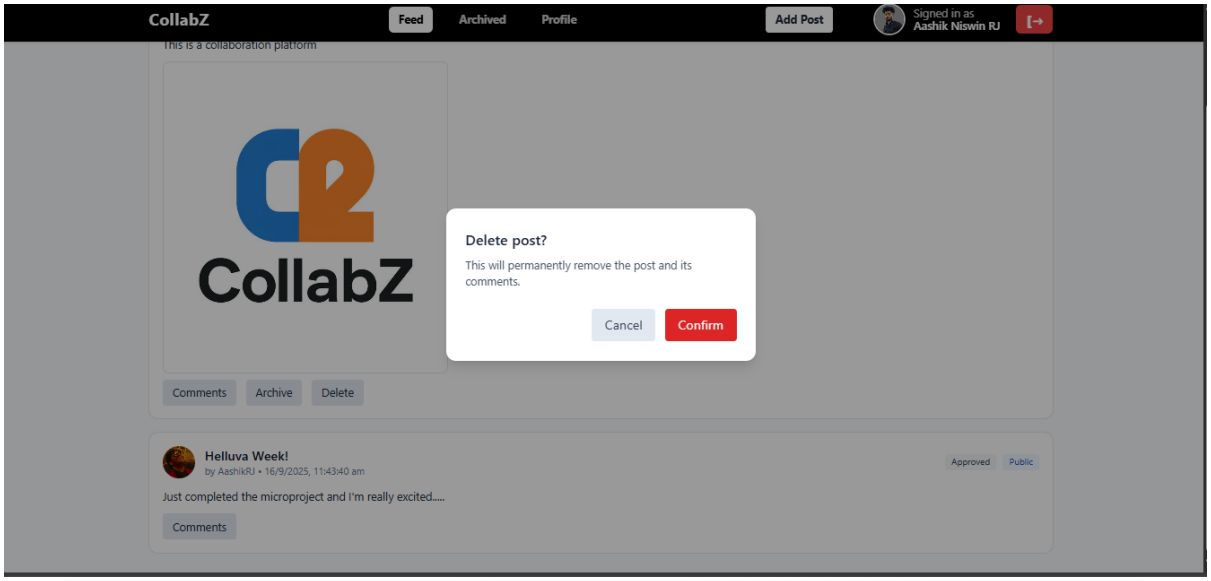
Pending Posts: (Admin Dashboard)



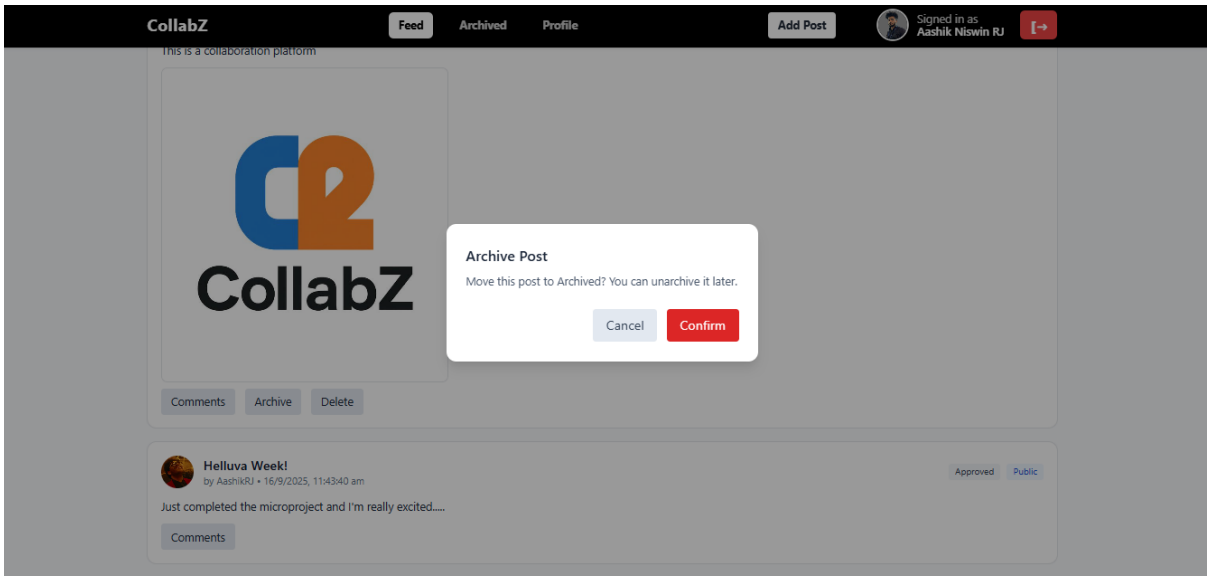
Feed Page (User):



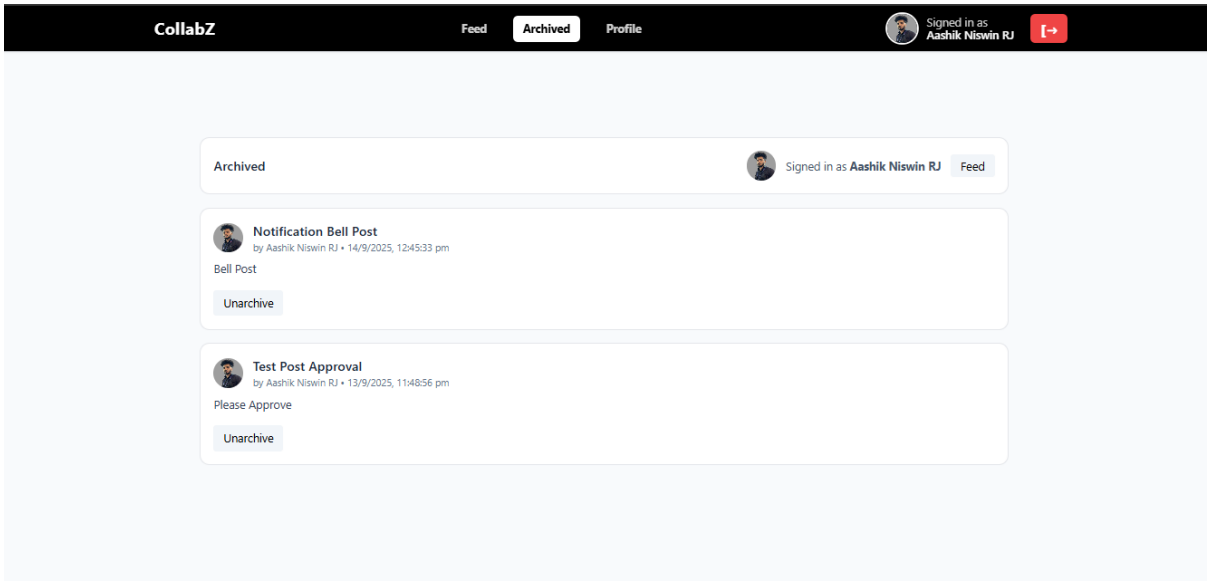
Delete Post:



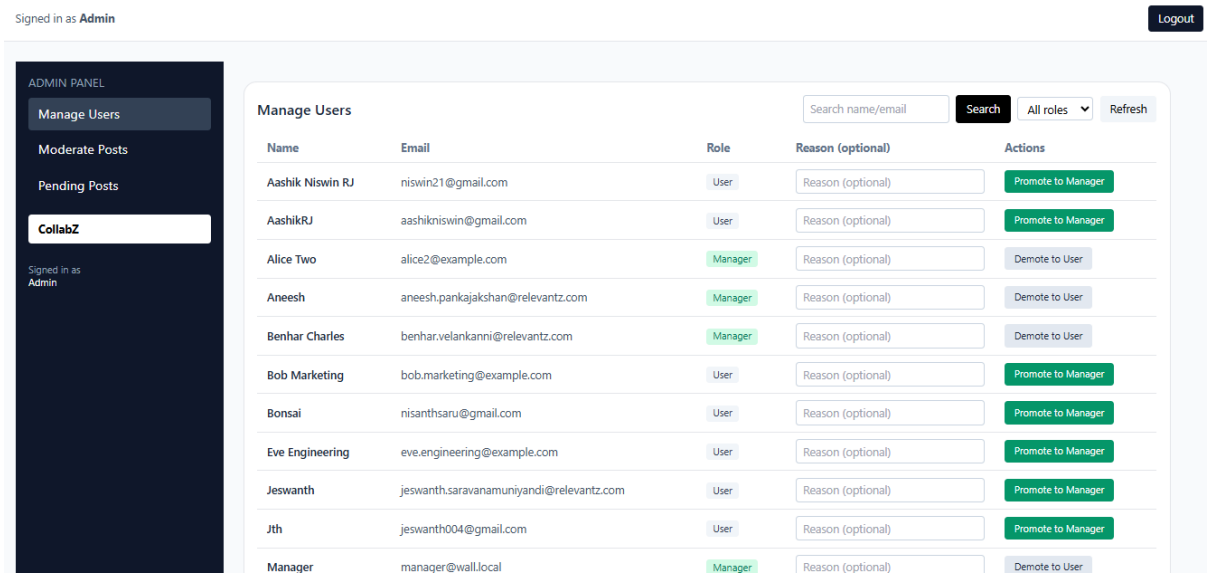
Archieve Post:



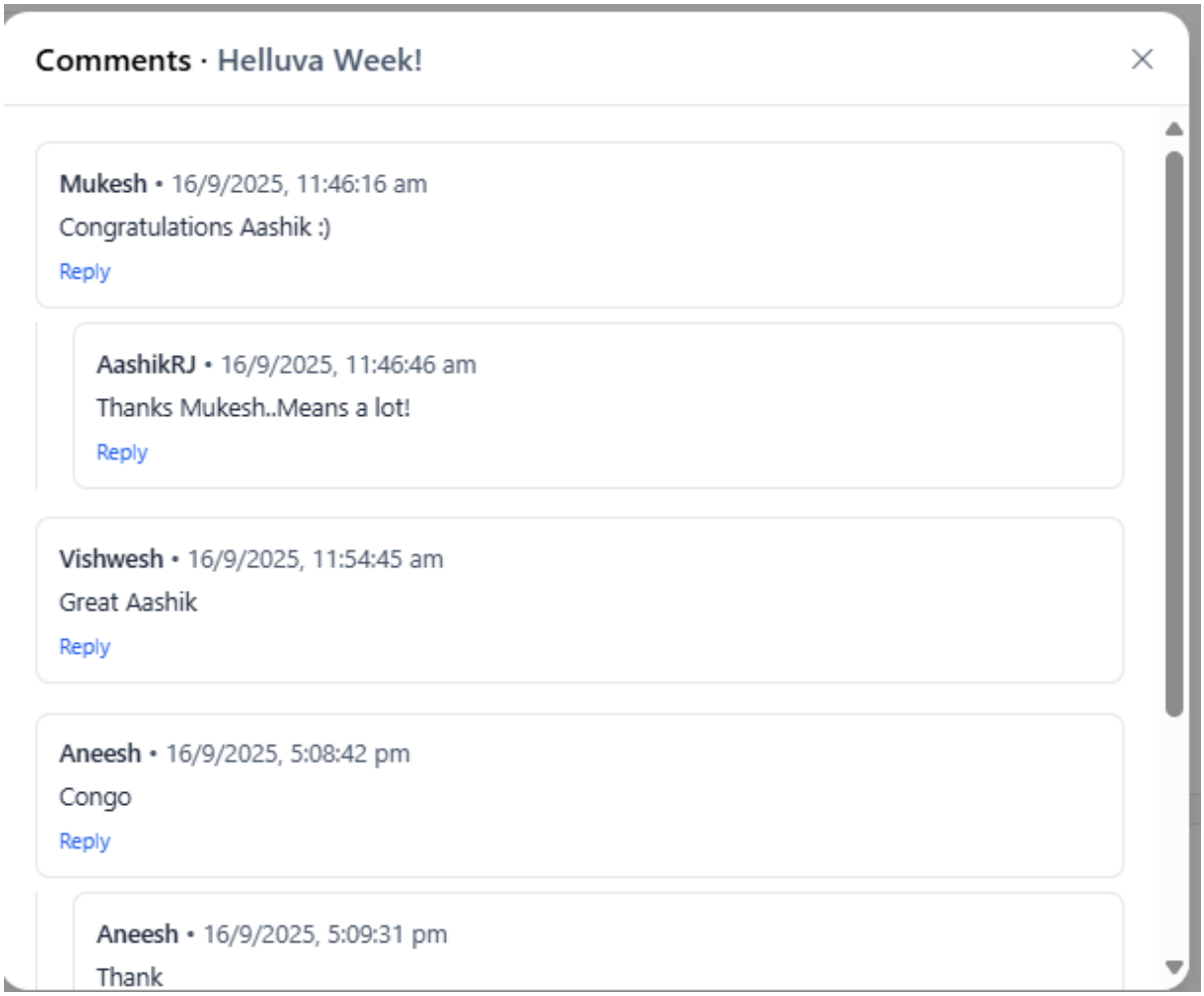
Archieved Posts:



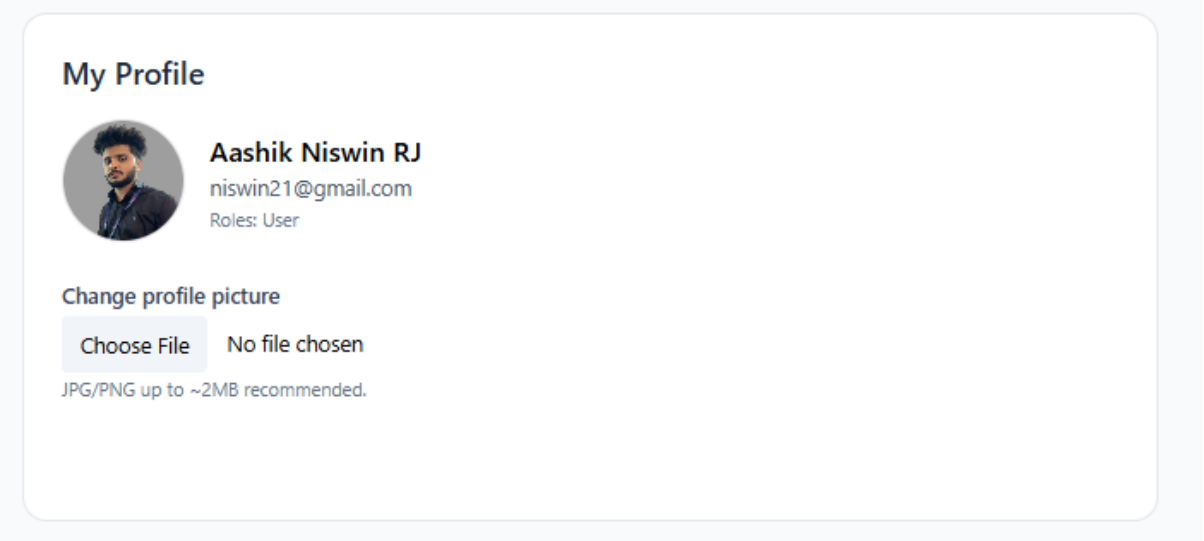
Admin Dashboard:



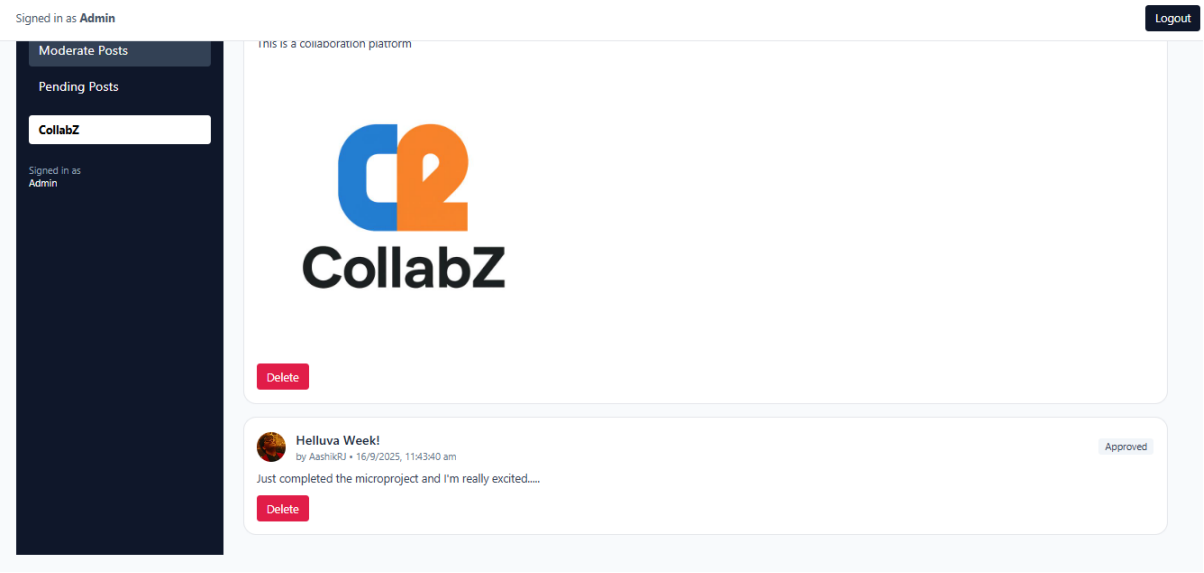
Comment Box:



Profile Page:

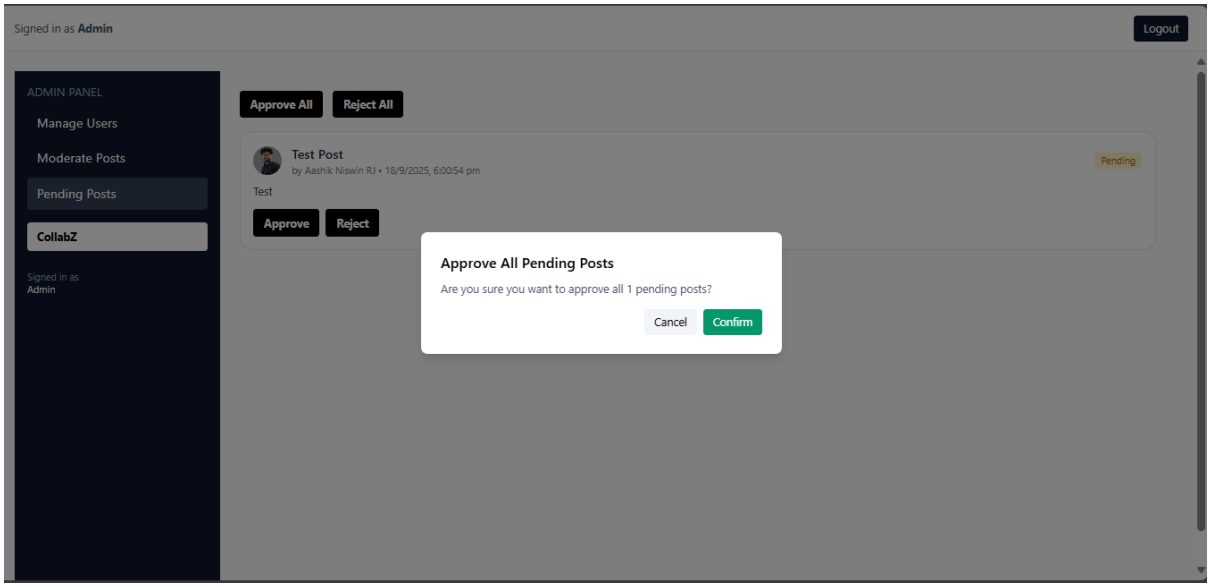


Moderate Posts (Admin Module) :

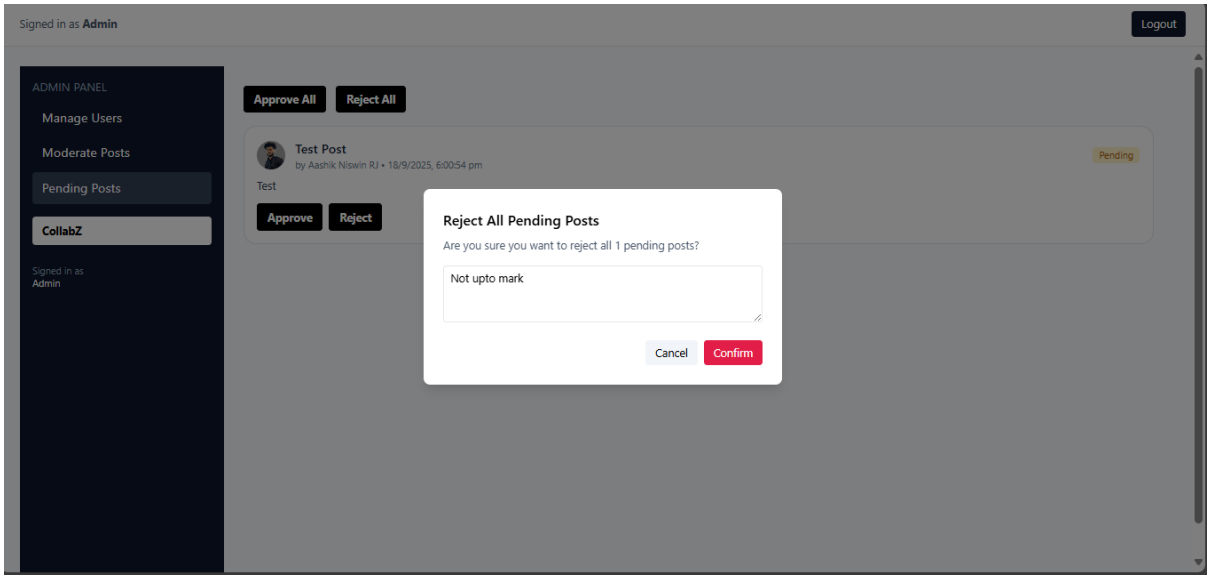


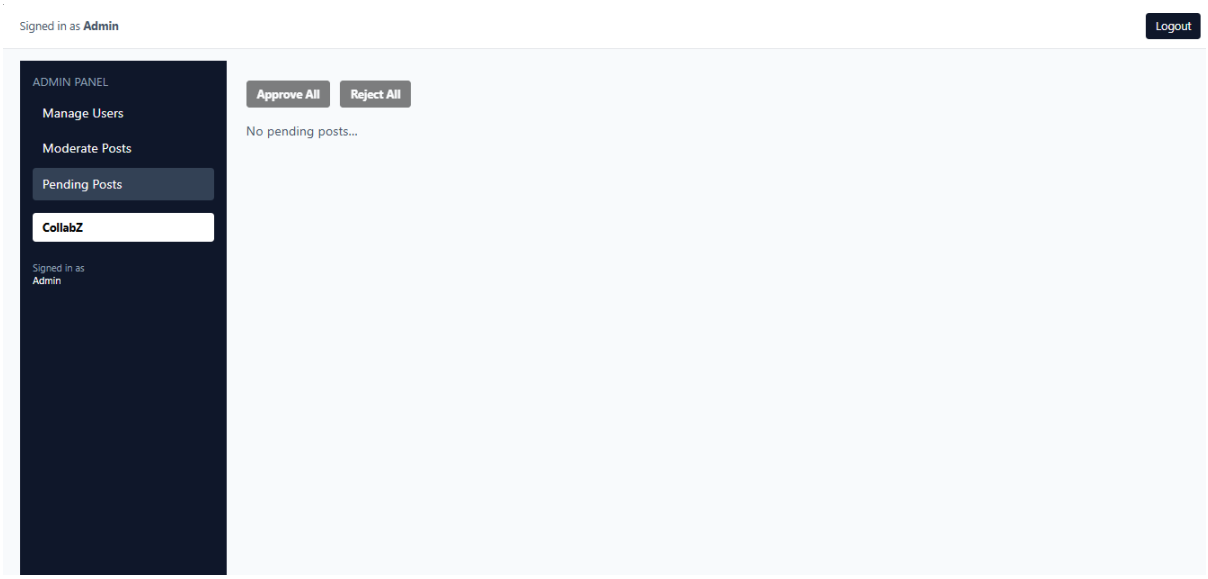
Pending Posts (Admin Module):

Approve All:

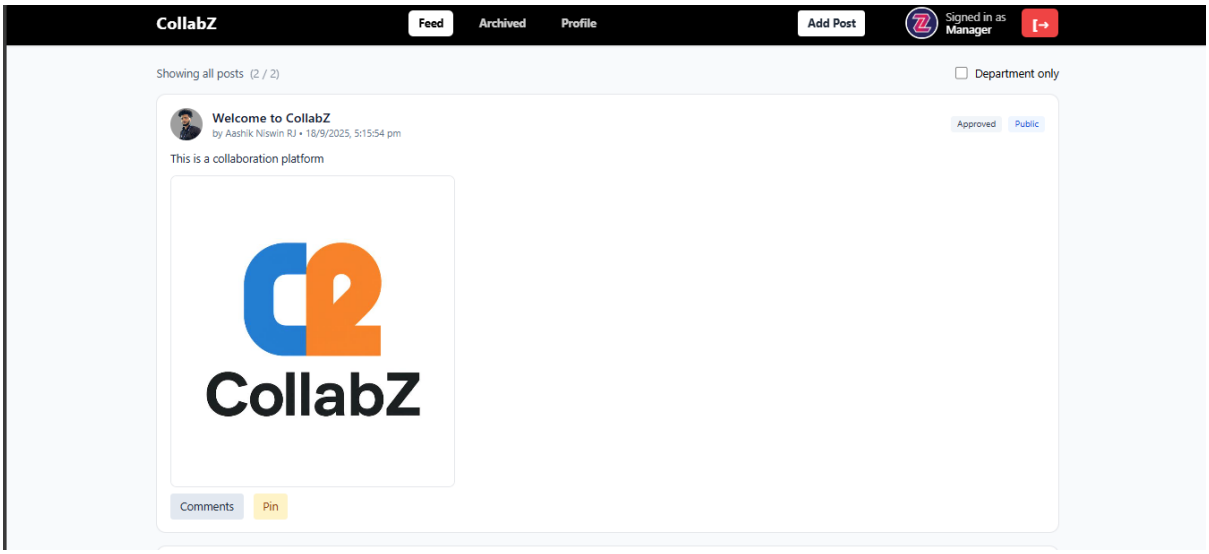

Signed in as Admin
Logout
ADMIN PANEL
Manage Users
Moderate Posts
Pending Posts
CollabZ
Signed in as
Admin
Approve All
Reject All
Test Post
by Aashik Niswin RJ • 18/9/2025, 6:00:54 pm
Pending
Test
Approve
Reject
Approve All Pending Posts
Are you sure you want to approve all 1 pending posts?
Cancel
Confirm

Reject All:

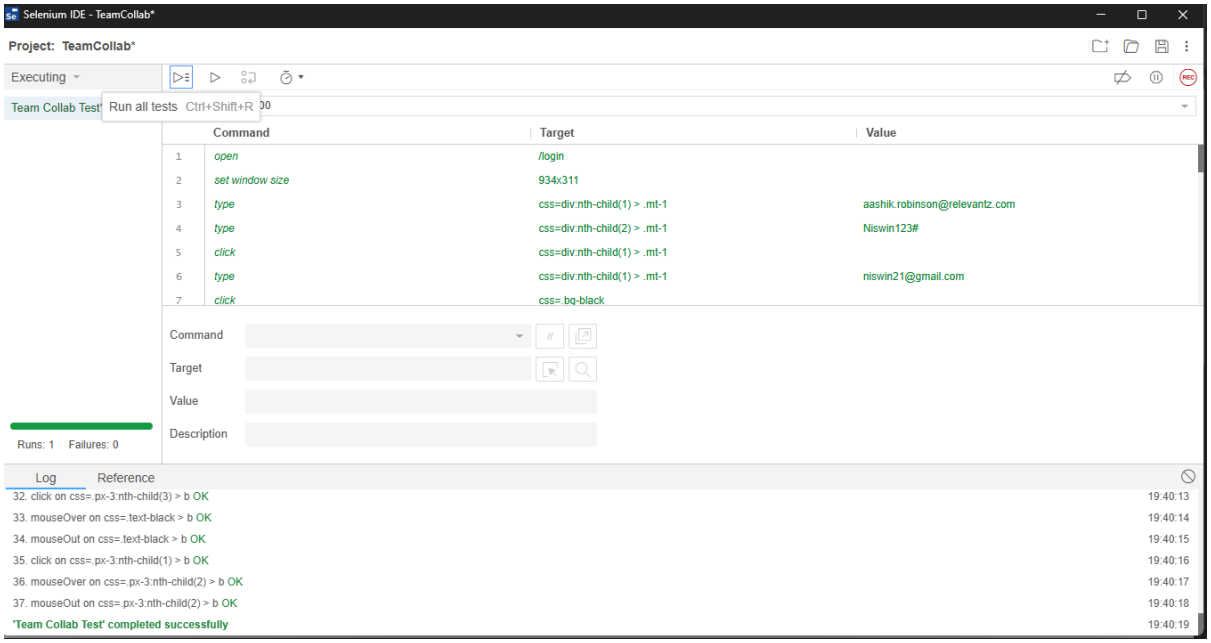

Signed in as Admin
Logout
ADMIN PANEL
Manage Users
Moderate Posts
Pending Posts
CollabZ
Signed in as
Admin
Approve All
Reject All
Test Post
by Aashik Niswin RJ • 18/9/2025, 6:00:54 pm
Pending
Test
Approve
Reject
Reject All Pending Posts
Are you sure you want to reject all 1 pending posts?
Not upto mark
Cancel
Confirm



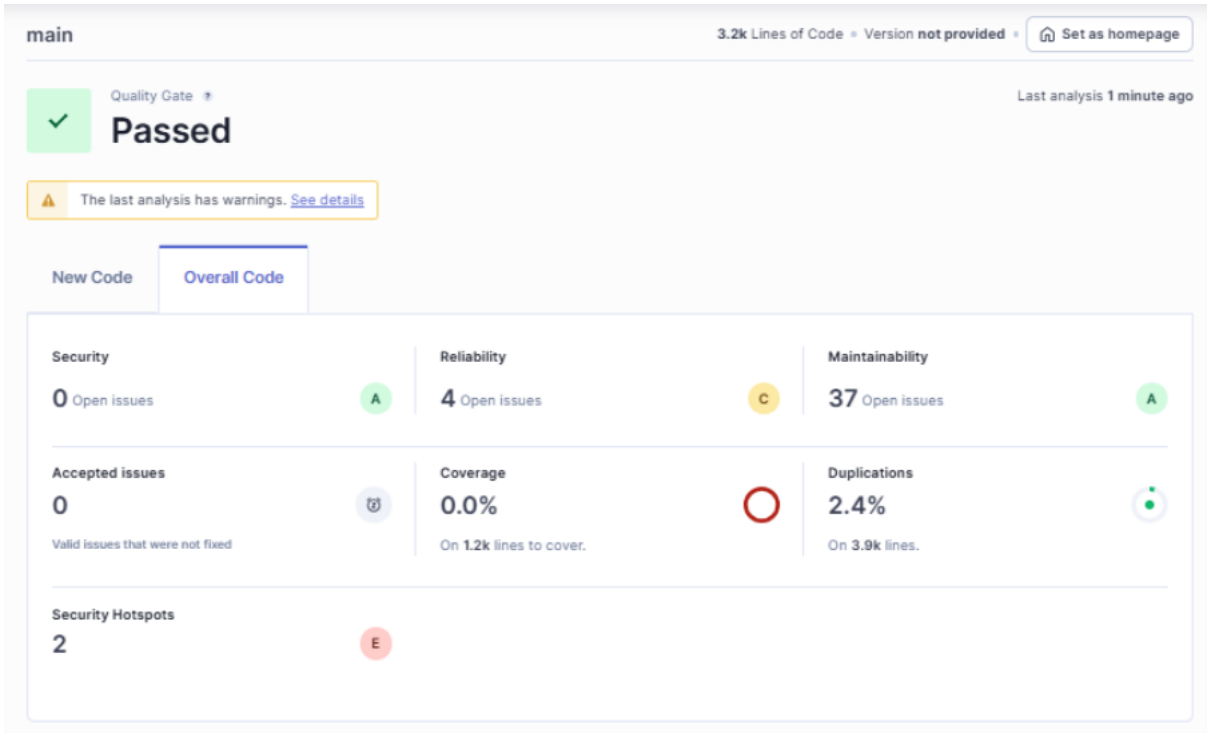
Manager Feed (Pin/Unpin Posts):



Selenium Testing:



SonarQube Report:



Jest Snapshot testing:

```
Snapshot Summary
 › 1 snapshot updated from 1 test suite.

Test Suites: 2 passed, 2 total
Tests:       5 passed, 5 total
Snapshots:   1 updated, 1 total
Time:        3.994 s
Ran all test suites related to changed files.

Watch Usage: Press w to show more.
```

```
Test Suites: 2 passed, 2 total
Tests:       5 passed, 5 total
Snapshots:   1 file obsolete, 1 written, 1 total
Time:        4.931 s
Ran all test suites.
```

CI/CD screenshot: